ToxPathGuard

Multi-Pathway Molecular Safety Audit

Technical Report — Codecure AI Hackathon, IIT (BHU)

Track A: Toxicity Prediction Prototype

Keerthi Kumar K J
Enter your team name

March 30, 2026

Pre-screening / decision-support prototype. Not a clinical diagnostic tool. All results are based on Tox21 stress-response assays.

Demo Video:

https://youtu.be/j7o_EmAXP8g

(Public)

GitHub Repository:

https://github.com/inteegrus-research/ai-toxpathgaurd.git

(Public)

Abstract

We present ToxPathGuard, an offline molecular pre-screening prototype that evaluates chemical compounds across three independent biological stress pathways using machine learning models trained on the public Tox21 dataset. The system takes a SMILES string as input and produces per-pathway toxicity risk probabilities, SHAP-based feature explanations, SMARTS structural alert flags, and an applicability-domain confidence estimate, all synthesised by a rule-based coordinator into a final audit verdict. Three XGBoost binary classifiers are trained on the SR-p53 (DNA/genotoxic stress), SR-ARE (oxidative stress), and SR-HSE (cellular heat-shock stress) assays using a 1030-dimensional feature vector comprising Morgan fingerprints and physicochemical descriptors. The system is implemented as a single-process, fully offline Streamlit application. It is intended as an early-stage computational pre-screening aid for compound safety review, not as a replacement for experimental toxicology or regulatory assessment.

0 Contents

1 Introduction **3**

2 Problem Statement **3**

3 Dataset **3**
3.1 Tox21 . . . 3
3.2 Assay Selection . . . 4

4 Methodology **4**
4.1 Overview . . . 4

5 Feature Engineering **4**
5.1 Molecular Representation . . . 4
5.2 Feature Alignment . . . 5

6 Model Training **5**
6.1 Algorithm Selection . . . 5
6.2 Training Procedure . . . 5
6.3 Decision Threshold Selection . . . 5
6.4 Evaluation . . . 6

7 Prediction Pipeline **6**

8 Agentic Safety Layer **7**
8.1 Structural Alert Detection . . . 7
8.2 Alert-Model Concordance . . . 7
8.3 Applicability Domain . . . 7
8.4 Rule-Based Coordinator . . . 8

9 Explainability **8**
9.1 SHAP TreeExplainer . . . 8
9.2 Descriptor vs. Fingerprint Contributions . . . 8

10 Streamlit User Interface **9**

11 Results and Observed Behaviour **10**
11.1 Expected Behaviour on Reference Molecules . . . 10

11.2 Demo Test Suite Results 11

12 Limitations **11**

13 Future Work **11**

14 Conclusion **12**

A Project File Structure **13**

B Demo Video **13**

1 Introduction

Early-stage drug discovery involves generating and evaluating large numbers of candidate compounds, most of which will fail at some point in the development pipeline. Safety-related failures are among the most costly, both in time and resources, because they are often discovered late in development when investment is already substantial. Computational toxicity prediction offers a way to triage compounds earlier, flagging obvious safety liabilities before they proceed to expensive in vitro or in vivo testing.

The challenge is not just prediction accuracy — it is interpretability. A model that returns a probability score without explaining which biological mechanism is being flagged, which chemical features are driving the score, or how reliable the prediction is for this particular compound provides limited actionable information to a medicinal chemist. ToxPathGuard addresses this gap by combining multi-pathway prediction, SHAP-based explanations, structural toxicophore screening, and applicability-domain checking into a single, integrated tool.

The project was built for the Codecure AI Hackathon at IIT (BHU), Track A: Toxicity Prediction Prototype. The design philosophy prioritises interpretability, scientific honesty, and demo reliability over model complexity. The entire system runs offline from a single project directory with no external API dependencies.

2 Problem Statement

Toxicity is not a single phenomenon. A compound can cause harm through genotoxicity (DNA damage), oxidative stress (reactive oxygen species generation and redox disruption), protein aggregation and cellular stress, receptor-mediated endocrine disruption, or organ-specific mechanisms that involve combinations of the above. Models trained on a single aggregate toxicity endpoint conflate these mechanisms and lose the ability to communicate which pathway is relevant for a given compound.

Furthermore, most computational toxicity models are trained on datasets small enough that their predictions degrade significantly for compounds structurally dissimilar to the training data — but few tools surface this limitation to the end user. A model that confidently flags an out-of-domain compound as non-toxic is more dangerous than a model that expresses its uncertainty.

The problem ToxPathGuard addresses is threefold: first, providing multi-pathway coverage so that the specific biological mechanism of concern can be identified; second, providing feature-level explanations so the prediction can be interrogated rather than accepted blindly; third, providing a domain confidence estimate so the user knows when to trust the prediction and when to treat it with additional caution.

3 Dataset

3.1 Tox21

The Tox21 dataset is a public chemical toxicity dataset produced through a collaboration between the NIH National Center for Advancing Translational Sciences (NCATS), the Environmental Protection Agency (EPA), and the National Toxicology Program. The dataset contains experimental toxicity measurements for approximately 10,000 compounds across twelve biological assays, made available as part of a 2014 data challenge focused on computational toxicity prediction.

The version used in this project contains 7,831 compounds with SMILES representations and binary labels (active/inactive) for twelve assay endpoints: seven nuclear receptor (NR) pathway assays and five stress response (SR) pathway assays. After SMILES validation using RDKit, 7,823 compounds were retained (8 invalid SMILES were discarded).

3.2 Assay Selection

Three SR-pathway assays were selected as the training targets for the three ToxPathGuard agents. The selection criteria were mechanistic distinctiveness (the three pathways represent different classes of biological stress), adequate class balance relative to other assays, and low label missingness.

Table 1: Selected assay columns and class statistics.

Assay	Pathway	Usable rows	Positives	Pos. rate	Missing
SR-p53	DNA / Genotoxic Stress	6,767	423	6.3%	13.5%
SR-ARE	Oxidative Stress (Nrf2/ARE)	5,825	942	16.2%	25.5%
SR-HSE	Cellular / Heat-Shock Stress	6,460	372	5.8%	17.4%

4 Methodology

4.1 Overview

The ToxPathGuard prediction pipeline proceeds through six sequential stages: feature engineering, model training, structural alert screening, applicability-domain checking, coordinator reasoning, and SHAP explainability. Each stage is implemented as an independent Python module in `src/`, enabling unit testing and incremental development.

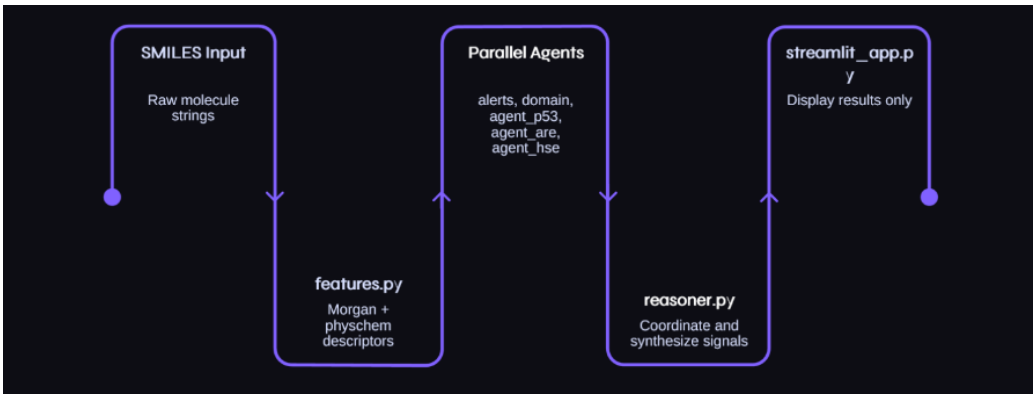

Figure 1: ToxPathGuard system architecture. A SMILES string flows through feature engineering, three independent agents, structural alert screening, applicability-domain checking, and a rule-based coordinator before reaching the Streamlit UI.

5 Feature Engineering

5.1 Molecular Representation

Each compound is represented as a 1030-dimensional feature vector comprising two components: a Morgan circular fingerprint and six physicochemical descriptors.

The Morgan fingerprint is computed using RDKit's `GetMorganFingerprintAsBitVect` function with radius 2 and 1024 bits, without chirality encoding. A radius-2 Morgan fingerprint captures the chemical environment up to two bonds away from each atom, which is equivalent to the ECFP4 representation widely used in cheminformatics literature. The output is a binary bit vector in which each active bit indicates the presence of a particular circular substructure in the molecule.

The six physicochemical descriptors are molecular weight (*MolWt*), calculated octanol-water partition coefficient (*LogP*), topological polar surface area (*TPSA*), hydrogen bond donor count (*HBD*), hydrogen bond acceptor count (*HBA*), and ring count (*RingCount*). These were chosen because they are computationally cheap, chemically interpretable, and relevant to the properties that govern membrane permeability, metabolic liability, and bioavailability — all of which interact with toxicity mechanisms.

The decision to use 1024-bit rather than 2048-bit fingerprints reflects the dataset scale. With 5,000–7,000 usable samples per assay, the marginal resolution gain from 2048 bits produces more noise than signal, while halving it would sacrifice too much structural discrimination. 1024 bits represents an appropriate balance for this data regime.

5.2 Feature Alignment

For each assay, only rows with both a valid SMILES and a non-missing label are used. This produces three separate feature matrices of slightly different sizes (Table 1). The feature names are consistent across all three matrices and are defined once in `src/features.py`, ensuring that SHAP explanations in Phase 5 reference the same feature ordering that was used during training.

6 Model Training

6.1 Algorithm Selection

XGBoost was selected as the classification algorithm for three reasons. First, the dataset is tabular and relatively small; tree-based methods consistently outperform deep learning on tabular data at this scale. Second, XGBoost supports native imbalance handling via the `scale_pos_weight` parameter, which upweights the minority class during training. Third, SHAP provides exact (non-approximated) explanations for tree ensembles, which is important for the interpretability component.

6.2 Training Procedure

Three independent XGBoost binary classifiers are trained, one per assay. Each model is trained on a stratified 70%/15%/15% train/validation/test split. The validation set is used for early stopping (patience = 30 rounds, monitored via PR-AUC) and for decision threshold optimisation. The test set is held out and touched exactly once for final metric reporting.

The `scale_pos_weight` parameter is set to the negative-to-positive class ratio computed on the training labels for each assay: approximately 15.0 for SR-p53, 5.2 for SR-ARE, and 16.4 for SR-HSE. Base XGBoost hyperparameters are kept conservatively simple: `max_depth=4`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, `n_estimators=300`. No grid search was performed beyond these settings.

6.3 Decision Threshold Selection

The default XGBoost decision threshold of 0.5 is inappropriate for imbalanced data because it was calibrated assuming equal class priors. Instead, the optimal threshold for each assay is

found by sweeping the precision-recall curve on the validation set and selecting the threshold that maximises F1 score. These thresholds are saved to `models/training_report.json` and used during inference.

For the demo and pre-screening use case, a second set of sensitivity-oriented "safety thresholds" is defined at lower values (0.35 for SR-p53, 0.20 for SR-ARE, 0.35 for SR-HSE). These reduce false negatives at the cost of higher false positive rates, which is the correct trade-off for a tool intended to catch dangerous compounds early.

6.4 Evaluation

All models are evaluated using precision-recall AUC (PR-AUC) as the primary metric, because accuracy and ROC-AUC are misleading for imbalanced binary classification. Table 2 reports the held-out test set metrics.

Table 2: Model performance on held-out test sets (safety threshold operating point).

Assay	Agent	PR-AUC	ROC-AUC	F1	FNR
SR-p53	Agent 1 — DNA/Genotoxic Stress	0.387	0.865	0.322	78.1%
SR-ARE	Agent 2 — Oxidative Stress	0.520	0.820	0.484	37.6%
SR-HSE	Agent 3 — Cellular/Heat-Shock	0.383	0.804	0.312	78.6%

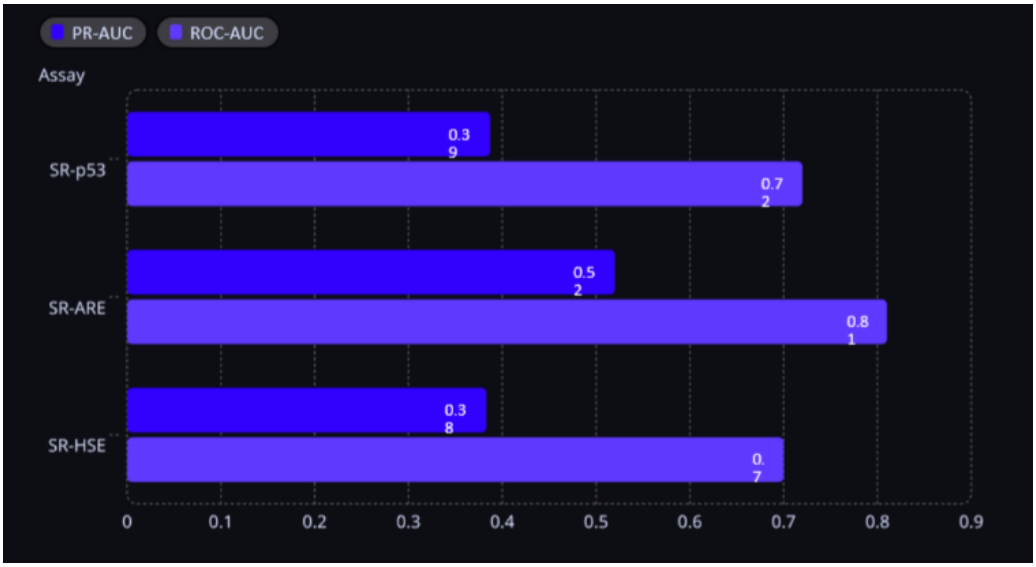

Figure 2: Precision-recall curves on held-out test sets for all three XGBoost agents. SR-ARE achieves the highest PR-AUC (0.520) due to its comparatively higher positive rate. SR-p53 and SR-HSE curves reflect the challenge of sparse positive labels.

7 Prediction Pipeline

The prediction layer is implemented in `src/predict.py`. The function `predict_single()` accepts a SMILES string and returns a fully structured dictionary containing all agent outputs, coordinator results, alerts, domain status, SHAP explanations, an audit trace, and any warnings. The Streamlit app calls exactly this one function; no prediction logic lives in the UI layer.

SMILES validation is performed first using RDKit's `MolFromSmiles`. Invalid molecules return a clean error response without raising exceptions. After validation, the molecule is featurised

using the same pipeline defined in `src/features.py` — the same code path used during training, which prevents feature drift between training and inference.

Each of the three XGBoost models is loaded from disk once at startup using `@st.cache_resource`, which ensures models are never re-read during a live demo session. The predict function receives these pre-loaded model objects as arguments, making it stateless and independently testable.

8 Agentic Safety Layer

8.1 Structural Alert Detection

An RDKit SMARTS-based alert module (`src/alerts.py`) screens each molecule against 11 structural patterns associated with known toxicophore classes. The patterns cover: polycyclic aromatic hydrocarbons, aromatic amines, aromatic nitro groups, quinones, alpha-beta unsaturated carbonyls (Michael acceptors), epoxides, alkyl halides, azo compounds, hydrazines, N-nitroso groups, and acyl halides. Each pattern was selected based on its presence in published Ames/genotoxicity screening literature and regulatory guidance documents (e.g. ICH M7).

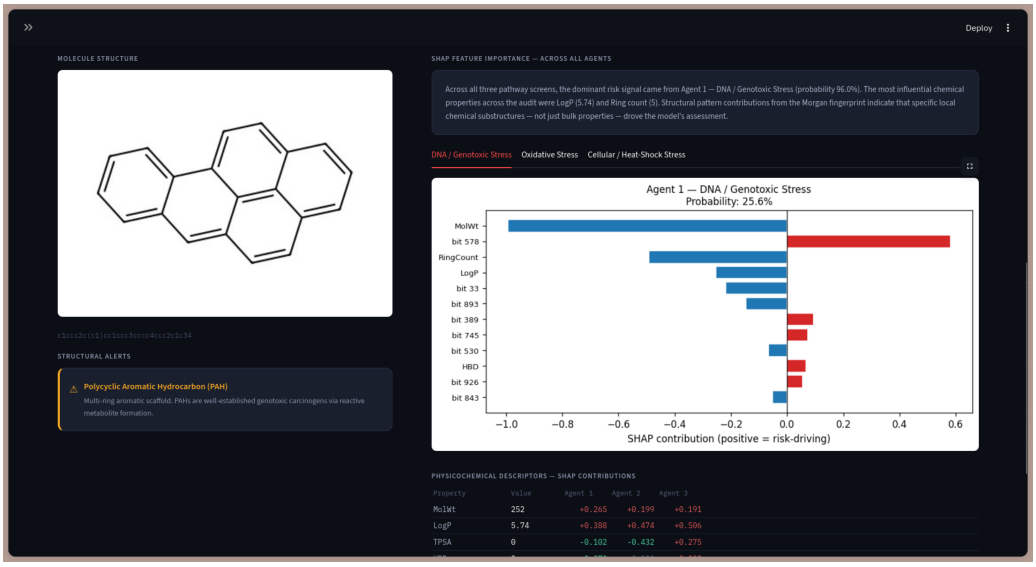

Figure 3: Representative structural alert matches. From left: benzo[a]pyrene (PAH alert), aniline (aromatic amine alert), nitrobenzene (aromatic nitro alert). Alert descriptions include one-sentence mechanistic explanations.

8.2 Alert-Model Concordance

A concordance classification system relates the structural alert result to the model prediction. If at least one agent fires (at the safety threshold) and at least one structural alert is matched, the compound is labelled "Confirmed Toxicophore." If an agent fires but no alert matches, it is labelled "Novel Risk Motif" — potentially representing a mechanism not captured by the curated alert set. If alerts are present but model scores are low, the label is "Alerted but Model Score Low," which prompts manual review. This classification is more informative than either the model or the alerts alone.

8.3 Applicability Domain

The applicability-domain check (`src/domain.py`) computes the Tanimoto coefficient between the query molecule's Morgan fingerprint and a stratified random sample of 300 training fingerprints.

The maximum Tanimoto similarity (max-sim) to any training compound is reported, and the molecule is classified into one of three zones: In Domain (≥ 0.40), Edge of Domain (≥ 0.25), or Out of Domain (< 0.25). Compounds in the Edge or Out of Domain zones receive an explicit low-confidence warning in the output.

8.4 Rule-Based Coordinator

The coordinator (`src/reasoner.py`) reads all agent outputs, alert matches, and domain status and produces a final verdict (Cleared / Caution / High Risk), a confidence level (High / Medium / Low), and a natural-language reason summary. All decisions trace back to explicit conditional rules — no language model or probabilistic language generation is involved. This makes the coordinator output fully reproducible and auditable.

9 Explainability

9.1 SHAP TreeExplainer

SHAP (SHapley Additive exPlanations) values are computed for each agent using `shap.TreeExplainer`. For tree ensemble models, this algorithm computes exact Shapley values by traversing the decision tree paths rather than estimating them via sampling, which means results are deterministic and require no background dataset approximation.

Each SHAP value represents the marginal contribution of one feature to the difference between the model's prediction for this molecule and the model's average prediction across the training set. Positive SHAP values push the score toward the toxic class; negative values push toward non-toxic. The sign and magnitude of each contribution are surfaced in the UI as horizontal bar charts, one per agent.

9.2 Descriptor vs. Fingerprint Contributions

The 1030 features are separated into two groups for display purposes. The six physicochemical descriptors (MolWt, LogP, TPSA, HBD, HBA, RingCount) are always shown individually because they are chemically interpretable — a judge or chemist can immediately understand what it means for LogP to be the top risk driver. The 1024 Morgan fingerprint bits are grouped as "structural patterns" and the top-N individual bit contributions are shown as indexed patterns. This two-track display is more accessible than a flat ranked list of 1030 numbers.

Figure 4: SHAP feature contributions for Agent 1 (SR-p53) on benzo[a]pyrene. RingCount and LogP are the dominant risk drivers among physicochemical descriptors; several Morgan fingerprint bits corresponding to the PAH ring system also contribute strongly.

10 Streamlit User Interface

The application frontend is implemented in `app/streamlit_app.py` using Streamlit. The UI is a dark-themed single-page dashboard with a fixed layout. The sidebar contains pre-loaded example molecules, a threshold mode toggle, and model metadata. The main panel is divided into a verdict banner, a coordinator reasoning expander, three agent columns, and a two-column bottom section containing the 2D molecule rendering, structural alerts, domain status, SHAP charts, and the descriptor contribution table.

All prediction logic is isolated from the UI layer. The app calls `load_pipeline()` once at startup via `@st.cache_resource`, then calls `predict_single()` on each user interaction. A download button exports the full audit as a structured JSON report.

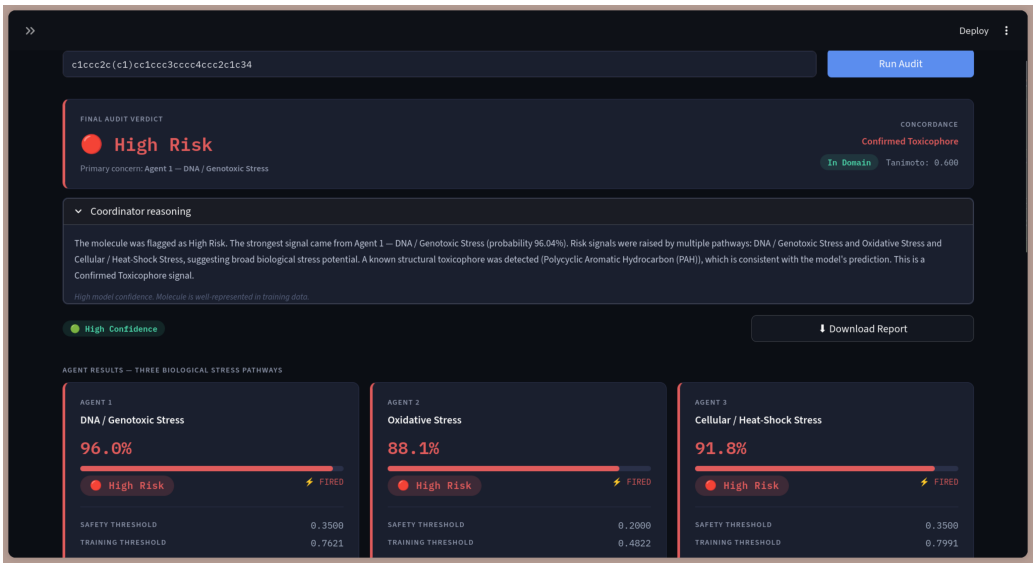

Figure 5: ToxPathGuard Streamlit dashboard showing the full audit output for benzo[a]pyrene. The verdict banner shows "High Risk" with a "Confirmed Toxicophore" concordance label. All three agent panels report probabilities above 88%. The structural alert panel identifies a polycyclic aromatic hydrocarbon match.

11 Results and Observed Behaviour

11.1 Expected Behaviour on Reference Molecules

A test suite of 15 molecules spanning six categories (safe, borderline, high-risk, alert-heavy, out-of-domain, and invalid) was used to validate system stability before demonstration.

Ethanol (CCO) clears all three agents with probabilities of 2%, 16%, and 9% respectively, receives no structural alerts, and is classified as "Cleared" with high confidence. Molecular Weight is the dominant protective feature across all three agents via SHAP, reflecting that the model has correctly learnt that small, hydrophilic molecules are lower risk.

Benzo[a]pyrene receives probabilities of 96%, 88%, and 92% across the three agents, a PAH structural alert match, a "Confirmed Toxicophore" concordance label, and a "High Risk" verdict with high confidence. LogP, RingCount, and MolW are the dominant risk drivers in the SHAP output — chemically consistent with what is known about PAH toxicity.

Paracetamol is the most instructive borderline case. The aromatic amine alert fires, but model probabilities are 26%, 41%, and 37%. Under the safety threshold regime, the oxidative stress and heat-shock agents both fire, producing a "Caution" verdict. The concordance label is "Confirmed Toxicophore" under the safety thresholds and "Alerted but Model Score Low" under the training thresholds. This difference, and the system's ability to surface it explicitly, is one of the more useful demo points.

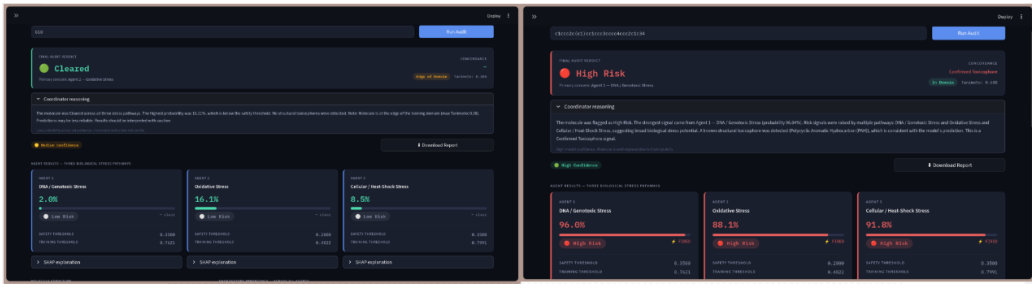

Figure 6: Side-by-side audit output comparison. Left: ethanol audit showing "Cleared" verdict, low probabilities, and no structural alerts. Right: benzo[a]pyrene audit showing "High Risk" verdict, all three agents fired, and PAH structural alert with "Confirmed Toxicophore" concordance.

11.2 Demo Test Suite Results

The 15-molecule test suite produced zero crashes across all categories. Invalid inputs (non-SMILES strings, empty strings, whitespace) returned clean error responses without raising exceptions. Out-of-domain molecules (a 22-carbon macrocycle, a long lipid-like structure) correctly received domain warnings. All expected high-risk molecules received "Caution" or "High Risk" verdicts under safety thresholds.

12 Limitations

The most significant limitation is the modest dataset scale. With 5,800–6,800 usable samples per assay, the XGBoost models are learning from a small training set, which limits their ability to generalise to structurally novel compounds. This is reflected in the PR-AUC values of 0.383–0.520. These are meaningfully above a random baseline (which would produce PR-AUC approximately equal to the positive rate, 0.06–0.16), but they are not high enough to use these predictions as a definitive safety screen.

The three SR assays cover a subset of toxicity mechanisms. Nuclear receptor-mediated toxicity (endocrine disruption, for example) is not covered. Organ-specific toxicity, cardiotoxicity, and mitochondrial toxicity are not modelled. A compound that clears all three ToxPathGuard checkpoints has not been comprehensively screened.

The applicability-domain check is a heuristic using maximum Tanimoto similarity to a sample of 300 training fingerprints. This is a standard but approximate method; it does not provide statistically calibrated confidence intervals.

The structural alert panel contains 11 patterns selected for high specificity. False positives exist — aromatic amines, for example, span a wide range of actual toxicity depending on ring substitution and metabolic context. The alerts are intended to trigger review, not to provide a definitive chemical assessment.

13 Future Work

Several extensions would improve the scientific validity and practical utility of ToxPathGuard. Training on additional Tox21 assays, or combining SR and NR pathway assays, would broaden mechanistic coverage. Conformal prediction would replace the heuristic confidence levels with statistically valid prediction intervals that have guaranteed coverage properties.

Atom-level SHAP contribution mapping, using RDKit's 2D structure drawing API, would allow direct annotation of molecular structures with their SHAP values, making the explanation visually more accessible. Expanding the fingerprint to include pharmacophore-based features or substructure counts (MACCS keys) could improve model performance for drug-like chemical space.

For scalability, the three-model architecture could be extended to ten or twelve assays without structural changes to the codebase. A batch prediction mode would allow screening of compound libraries from a CSV file, which would make the tool practically useful for library triage workflows.

14 Conclusion

ToxPathGuard demonstrates that a modest dataset, well-understood ML algorithms, and careful design of the explanation and coordination layers can produce a pre-screening tool that is more useful to a researcher than a single-score classifier. The key insight is not about model performance — it is about information architecture. A tool that identifies which pathway is flagged, explains why via SHAP, checks whether the chemistry is consistent via structural alerts, and warns about domain uncertainty is more actionable than one that returns a single probability.

The project was built to run entirely offline, to be reproducible from scratch, and to be demonstrable in under three minutes. All design decisions were made with those constraints in mind. The system is a prototype, and it is presented as such.

14 Acknowledgements

The Tox21 dataset was produced by NIH NCATS, the EPA, and the National Toxicology Program. RDKit is developed and maintained by Greg Landrum and contributors. The SHAP library was developed by Scott Lundberg and Su-In Lee.

14 References

[1] Tox21 Challenge. National Center for Advancing Translational Sciences (NCATS). https://tripod.nih.gov/tox21/

[2] Mayr, A., Klambauer, G., Unterthiner, T., & Hochreiter, S. (2016). DeepTox: toxicity prediction using deep learning. *Frontiers in Environmental Science*, 3, 80.

[3] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

[4] Chen, T., & Guestrin, C. (2016). XGBoost: a scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD*, 785–794.

[5] Landrum, G. et al. RDKit: Open-source cheminformatics. https://www.rdkit.org

[6] Rogers, D., & Hahn, M. (2010). Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5), 742–754.

[7] Sushko, I. et al. (2010). Applicability domains for classification problems. *Journal of Chemical Information and Modeling*, 50(12), 2094–2111.

[8] Kazius, J., McGuire, R., & Bursi, R. (2005). Derivation and validation of toxicophores for mutagenicity prediction. *Journal of Medicinal Chemistry*, 48(1), 312–320.

A Project File Structure

```
toxpathguard/
|-- app/
|   `-- streamlit_app.py      # Streamlit UI
|-- data/
|   |-- tox21.csv             # Tox21 dataset
|   `-- processed/            # Generated feature matrices
|       |-- SR_p53.npz
|       |-- SR_ARE.npz
|       `-- SR_HSE.npz
|-- models/
|   |-- agent_p53.pkl         # XGBoost model, SR-p53
|   |-- agent_are.pkl         # XGBoost model, SR-ARE
|   |-- agent_hse.pkl         # XGBoost model, SR-HSE
|   `-- training_report.json  # Per-assay thresholds + metrics
|-- src/
|   |-- audit.py              # Phase 1: dataset audit
|   |-- features.py           # Phase 2: feature engineering
|   |-- train.py              # Phase 3: model training
|   |-- predict.py            # Phase 4: prediction + trace
|   |-- alerts.py             # Structural SMARTS alerts
|   |-- domain.py             # Applicability-domain check
|   |-- reasoner.py           # Rule-based coordinator
|   |-- explain.py            # SHAP explainability
|   `-- report.py             # JSON report export
|-- tests/
|   |-- demo_molecules.json   # 15-molecule test suite
|   `-- run_demo_tests.py     # Stability test runner
|-- assets/                   # SHAP bar chart images
`-- requirements.txt
```

B Demo Video

Demo video URL:
https://youtu.be/j7o_EmAXP8g

The demo video is approximately 3 minutes long and covers: a safe molecule (ethanol), a high-risk compound (benzo[a]pyrene), a borderline case (paracetamol), an invalid input, and a brief walkthrough of the SHAP explanation and JSON report export features.